PROJECT REPORT

STUDENT MARKS ANALYZER USING PYTHON

Introduction to AI and Python Essentials

Submitted by	Tanisha Sharma
Roll number	USN NO 251VMTR1365
College / Institute	Jain Deemed University
Course	Introduction to AI and Python Essentials
Submission date	2026-10-22

A beginner Python project demonstrating input/output, arithmetic, conditional logic, input validation, and rule-based recommendations.

1. Abstract

The Student Marks Analyzer is a command-line application written in Python. It collects a student's name and marks in five subjects, checks that each mark is between 0 and 100, calculates the total and average, assigns a grade, and displays a study suggestion.

The project is suitable for a beginner course because it combines basic programming concepts with a simple rule-based decision system. The program follows transparent rules written by the programmer. It is not a trained machine-learning model.

2. Introduction

Python is a high-level programming language known for readable syntax and broad use in automation, data analysis, and artificial intelligence. Many AI systems use data and decision logic to produce outputs. This project introduces a small example of automated analysis: the program calculates marks and interprets the average consistently.

3. Problem Statement

Calculating totals, averages, grades, and study suggestions manually can take time and may lead to arithmetic errors. A simple program can perform these repeated calculations and apply the same grading rules each time.

4. Objectives

- Accept the student's name and five subject marks.
- Reject invalid marks and non-numeric input.
- Calculate total marks out of 500 and average percentage.
- Assign a grade using clear thresholds.
- Display a rule-based suggestion for improvement.

5. Tools and Requirements

- Language: Python 3
- Interface: Command line / terminal
- Required libraries: Python standard library only
- Platform: Windows, macOS, or Linux

6. Programming Concepts Used

Input and output: input() collects the name and marks; print() displays results.
Variables and lists: marks are stored in a list and calculations in variables.
Loops: for repeats the mark-entry step five times; while repeats input until a valid mark is entered.
Exception handling: try/except handles non-numeric input safely.
Conditional statements: if/elif/else select a grade and recommendation.
Functions: get_mark(), analyze_performance(), and main() separate tasks.

7. Algorithm

1. Start the program.
2. Ask for the student's name; use “Student” if the name is blank.
3. Repeat five times: request a mark, ensure it is numeric and within 0–100, and store it.
4. Add the marks to calculate the total.
5. Divide the total by five to calculate the average.
6. Compare the average with grade thresholds.
7. Display the total, average, grade, and suggestion.
8. End.

8. Grading Rules

Average	Grade	General interpretation
90–100	A+	Excellent performance
80–89.99	A	Very good performance
70–79.99	B	Good performance
60–69.99	C	Fair performance
40–59.99	D	Basic threshold reached
Below 40	F	Focus on fundamentals

9. Source Code

The full runnable source code is included as **student_marks_analyzer.py** in this submission. The main logic is shown below.

```
"""
Project: Student Marks Analyzer using Python
Student: Tanisha Sharma
Course: Introduction to AI and Python Essentials

This beginner-friendly program analyzes marks for five subjects, calculates
the total and average, assigns a grade, and gives a rule-based study suggestion.

Note: This is a rule-based program inspired by simple decision systems. It is
not a machine-learning model and does not learn from data.
"""

def get_mark(subject_number):
    """Read and validate a mark from 0 to 100."""
    while True:
        try:
            mark = float(input(f"Enter marks for Subject {subject_number} (0-100): "))
            if 0 <= mark <= 100:
                return mark
            print("Please enter a mark between 0 and 100.")
        except ValueError:
            print("Invalid input. Please enter a number.")


def analyze_performance(average):
    """Return a grade and a rule-based recommendation."""
    if average >= 90:
        return "A+", "Excellent performance. Keep challenging yourself."
    elif average >= 80:
        return "A", "Very good work. Revise regularly to maintain your result."
    elif average >= 70:
        return "B", "Good performance. Practise the topics where you lost marks."
    elif average >= 60:
        return "C", "Fair performance. Make a weekly revision plan."
    elif average >= 40:
        return "D", "You passed the basic threshold. Practise more and ask for help."
    else:
        return "F", "Focus on fundamentals and make a step-by-step improvement plan."


def main():
    print("=" * 44)
    print("       STUDENT MARKS ANALYZER")
    print("=" * 44)

    name = input("Enter student name: ").strip()
    if not name:
        name = "Student"

    marks = []
    for subject_number in range(1, 6):
        marks.append(get_mark(subject_number))

    total = sum(marks)
    average = total / len(marks)
    grade, suggestion = analyze_performance(average)

    print("\n" + "-" * 44)
    print("               RESULT")
    print("-" * 44)
    print(f"Student name : {name}")
    print(f"Total marks  : {total:g} / 500")
    print(f"Average      : {average:.2f}%")
    print(f"Grade        : {grade}")
    print(f"Suggestion   : {suggestion}")
    print("-" * 44)


if __name__ == "__main__":
    main()
```

10. Sample Run

Example input: student name = Tanisha; marks = 85, 92, 88, 90, 85.

```
==========================================
       STUDENT MARKS ANALYZER
==========================================
Enter student name: Tanisha
Enter marks for Subject 1 (0-100): 85
Enter marks for Subject 2 (0-100): 92
Enter marks for Subject 3 (0-100): 88
Enter marks for Subject 4 (0-100): 90
Enter marks for Subject 5 (0-100): 85

------------------------------------------
               RESULT
------------------------------------------
Student name : Tanisha
Total marks  : 440 / 500
Average      : 88.00%
Grade        : A
Suggestion   : Very good work. Revise regularly to maintain your result.
------------------------------------------
```

11. Testing and Validation

Test case	Expected behavior
All five marks valid	Total and average are calculated; grade and suggestion appear.
Mark below 0 or above 100	Program displays a range warning and asks again.
Letters entered instead of a mark	Program displays an invalid-input message and asks again.
Blank student name	Program uses the default name “Student”.

12. Connection to AI

This project demonstrates a simple rule-based decision system: an input value (the average) is compared with predefined thresholds, and the program returns a grade and recommendation. The rules are transparent and easy to explain.

Important limitation: this program does not train a model, learn from examples, or make predictions from a dataset. It should be described as a beginner Python project with rule-based logic, not as a machine-learning application.

13. Limitations and Future Enhancements

Current limitations include a fixed number of five subjects, the same maximum mark for every subject, and hard-coded grade thresholds. Future versions could allow a variable number of subjects, save results to a CSV file, compare performance over time, or use a dataset for a separately designed machine-learning exercise.

14. Conclusion

The Student Marks Analyzer automates basic academic calculations and demonstrates fundamental Python skills, including input validation, loops, lists, functions, arithmetic, and conditional logic. It produces consistent results and gives a simple next-step suggestion based on the student's average.

15. References

• Python Software Foundation, Python 3 documentation: https://docs.python.org/3/
• Course materials for Introduction to AI and Python Essentials.

Submission checklist

• Personalized report with student details
• Python source file
• README with run instructions
• Code syntax and behavior checks completed before packaging